

ECE/CS 757 Advanced Computer Architecture II

Spring 2025

Transactional Memory (Part 2)

Instructor: Prof. Joshua San Miguel

Some slides adopted from Greg Steffan and Natalie Enright Jerger

Last Lecture

- Optimistic concurrency
- Version management and conflict detection

This Lecture

- Version management and conflict detection
- SW and HW transactional memory
- TM challenges

Transactional Memory

Version Management:

How do we track reads and writes to memory that may have potential conflicts?

- Eager vs. lazy

Conflict Detection:

How do we identify and resolve conflicts between multiple in-flight transactions?

- Eager vs. lazy

Conflict Detection

Need to roll back the transaction(s) if a conflict has occurred or if a future conflict may go undetected

➤ e.g., Possible outcomes of A and B?

```
// init
```

```
*x = 0; *y = 0;
```

```
// P1
```

```
atomic {
```

```
    *x = 1;
```

```
    *y = 1;
```

```
}
```

```
// P2
```

```
atomic {
```

```
    A = *x;
```

```
    B = *y;
```

```
}
```



Conflict Detection

Need to roll back the transaction(s) if a conflict has occurred or if a future conflict may go undetected

➤ e.g., Possible outcomes of A and B? (A=0 B=0) (A=1 B=1)

```
// init
```

```
*x = 0; *y = 0;
```

```
// P1
```

```
atomic {
```

```
    *x = 1;
```

```
    *y = 1;
```

```
}
```

```
// P2
```

```
atomic {
```

```
    A = *x;
```

```
    B = *y;
```

```
}
```



Conflict Detection

Need to roll back the transaction(s) if a conflict has occurred or if a future conflict may go undetected

➤ e.g., Possible outcomes of A and B? (A=0 B=0) (A=1 B=1)

```
// init
*x = 0; *y = 0;

// P1                                // P2
atomic {                             atomic {
  *x = 1; ← WAR                      A = *x;
  *y = 1; ← WAR                      B = *y;
}
```

Conflict Detection

Need to roll back the transaction(s) if a conflict has occurred or if a future conflict may go undetected

➤ e.g., Possible outcomes of A and B? (A=0 B=0) (A=1 B=1)

```
// init
```

```
*x = 0; *y = 0;
```

```
// P1
```

```
atomic {
```

```
  *x = 1;
```

```
  *y = 1;
```

```
}
```

```
// P2
```

```
atomic {
```

```
  A = *x;
```

```
  B = *y;
```

```
}
```

RAW

RAW



Conflict Detection

Need to roll back the transaction(s) if a conflict has occurred or if a future conflict may go undetected

➤ e.g., Possible outcomes of A and B? (A=0 B=0) (A=1 B=1)

```
// init
```

```
*x = 0; *y = 0;
```

```
// P1
```

```
atomic {
```

```
  *x = 1;
```

```
  *y = 1;
```

```
}
```

```
// P2
```

```
atomic {
```

```
  A = *x;
```

```
  B = *y;
```

```
}
```

transactions should not be
interdependent



Conflict Detection

Need to roll back the transaction(s) if a conflict has occurred or if a future conflict may go undetected

- e.g., Possible outcomes of A and B? (A=0 B=0) (A=1 B=1) (A=0 B=1)
not atomic

```
// init
```

```
*x = 0; *y = 0;
```

```
// P1
```

```
atomic {
```

```
  *x = 1;
```

```
  *y = 1;
```

```
}
```

```
// P2
```

```
atomic {
```

```
  A = *x;
```

```
  B = *y;
```

```
}
```

WAR

RAW



Conflict Detection

Need to roll back the transaction(s) if a conflict has occurred or if a future conflict may go undetected

- e.g., Possible outcomes of A and B? (A=0 B=0) (A=1 B=1) (A=1 B=0)
not atomic

```
// init
*x = 0; *y = 0;

// P1                                // P2
atomic {                             atomic {
  *x = 1;                             A = *x;
  *y = 1;                             B = *y;
}                                     }
```

Diagram illustrating a conflict between two processors (P1 and P2) accessing shared memory variables (*x and *y) using atomic operations. P1 writes to *x and *y, while P2 reads from *x and *y. The diagram shows a **RAW** (Read After Write) conflict for *x and a **WAR** (Write After Read) conflict for *y.

Conflict Detection

Need to roll back the transaction(s) if a conflict has occurred or if a future conflict may go undetected

- **Eager:** Check for conflicts at every load and store
- **Lazy:** Check for conflicts at commit time

Eager Conflict Detection – Eager VM

Upon committing transaction:

Clear Write Set and Read Set

Eager Conflict Detection – Eager VM

Upon committing transaction:

Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

Eager Conflict Detection – Eager VM

Upon committing transaction:

Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

➤ If X is in T's Read Set: **roll back?**

my {	T {
*A = 1;	... = *A;
...	...

Eager Conflict Detection – Eager VM

Upon committing transaction:

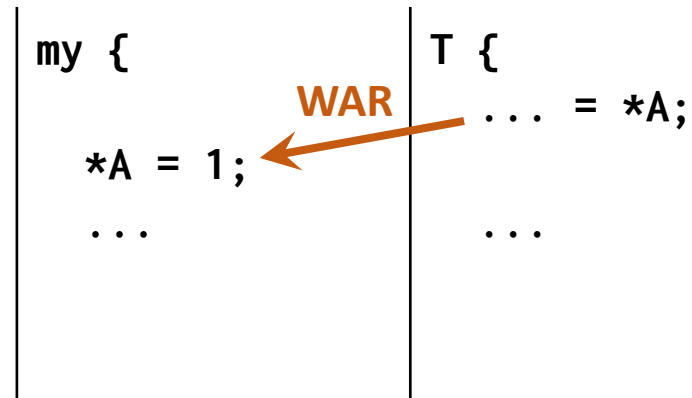
Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

➤ If X is in T's Read Set: **roll back?**



Eager Conflict Detection – Eager VM

Upon committing transaction:

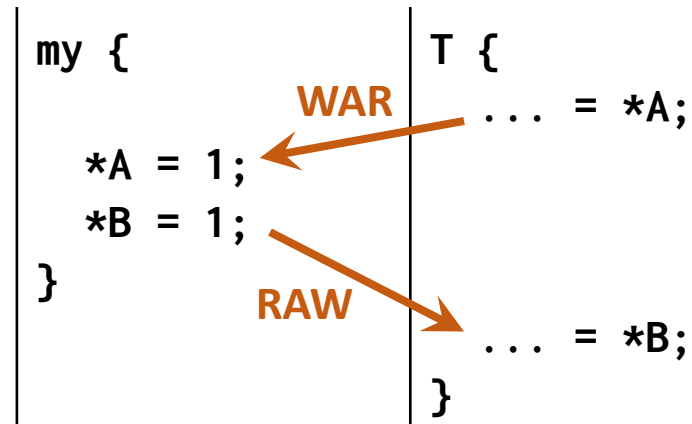
Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

➤ If X is in T's Read Set: **roll back?**



Eager Conflict Detection – Eager VM

Upon committing transaction:

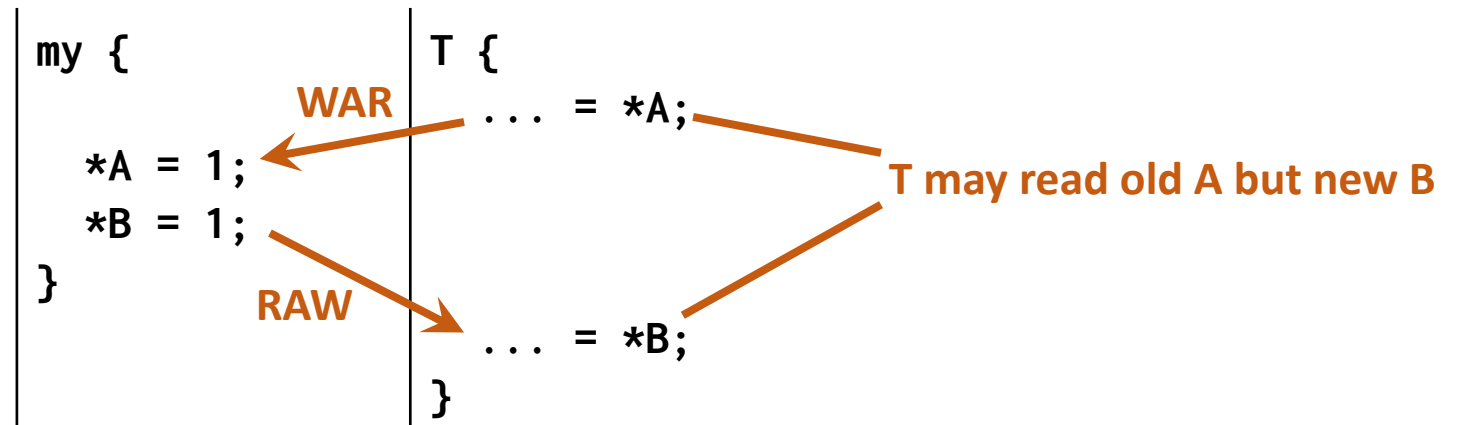
Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

➤ If X is in T's Read Set: **roll back either transaction**



Eager Conflict Detection – Eager VM

Upon committing transaction:

Clear Write Set and Read Set

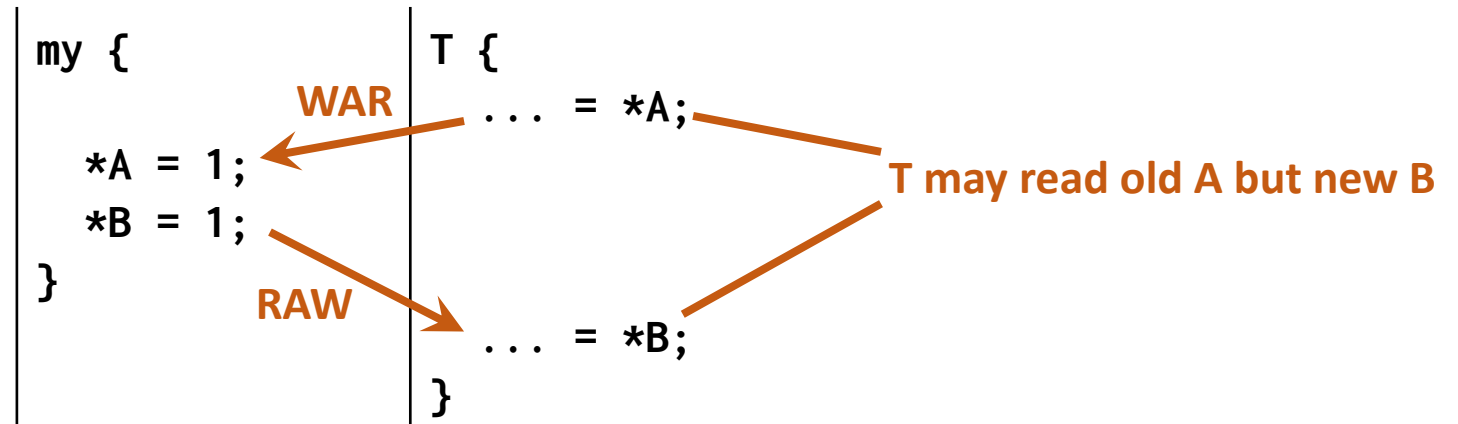
Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

➤ If X is in T's Read Set: **roll back either transaction**

balance between evicting transaction that
started later vs. is further from finish line



Eager Conflict Detection – Eager VM

Upon committing transaction:

Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

- If X is in T's Read Set: roll back either transaction
- If X is in T's Write Set: **roll back either transaction?**

```
my {  
  
    *A = 0;  
    ...  
}
```

```
T {  
    *A = 1;  
    ...  
}
```



Eager Conflict Detection – Eager VM

Upon committing transaction:

Clear Write Set and Read Set

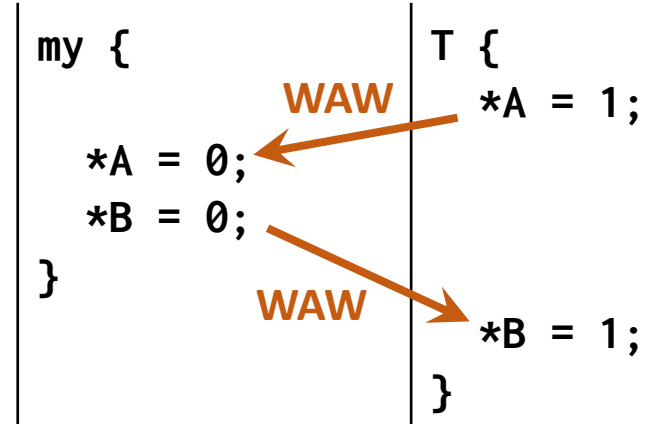
memory may end up with
my A but T's B

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

- If X is in T's Read Set: roll back either transaction
- If X is in T's Write Set: **roll back either transaction**



Eager Conflict Detection – Eager VM

Upon committing transaction:

Clear Write Set and Read Set

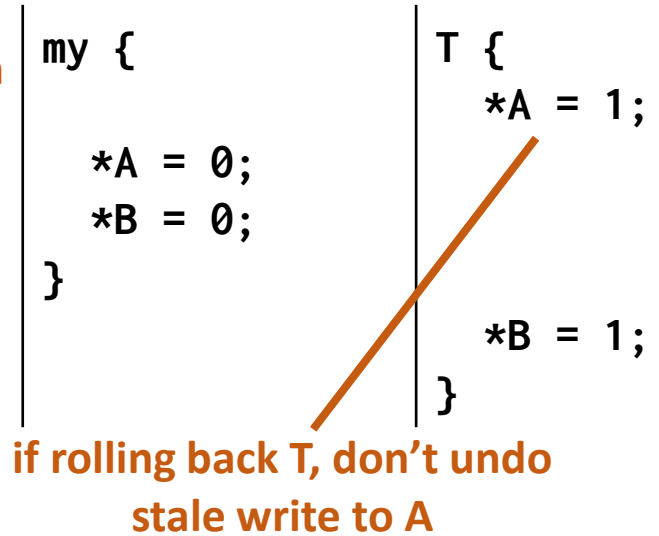
memory may end up with
my A but T's B

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

- If X is in T's Read Set: roll back either transaction
- If X is in T's Write Set: **roll back either transaction**



Eager Conflict Detection – Eager VM

Upon committing transaction:

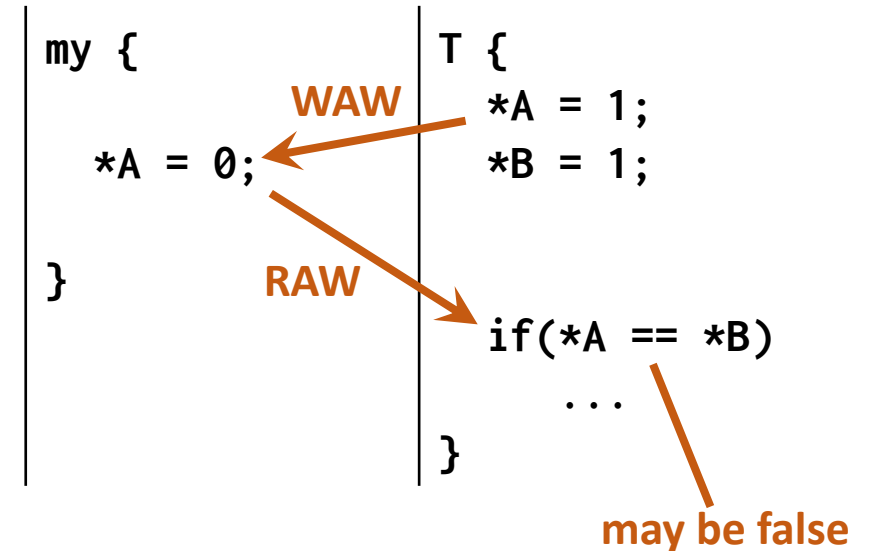
Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

- If X is in T's Read Set: roll back either transaction
- If X is in T's Write Set: **roll back either transaction**



Eager Conflict Detection – Eager VM

Upon committing transaction:

Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

- If X is in T's Read Set: roll back either transaction
- If X is in T's Write Set: roll back either transaction

Upon a load instruction:

Add address X to Read Set (if not already there)

For each in-flight transaction T:

- If X is in T's Write Set: **roll back either transaction?**

my {	T {
...	*A = 1;
...	...

Eager Conflict Detection – Eager VM

Upon committing transaction:

Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

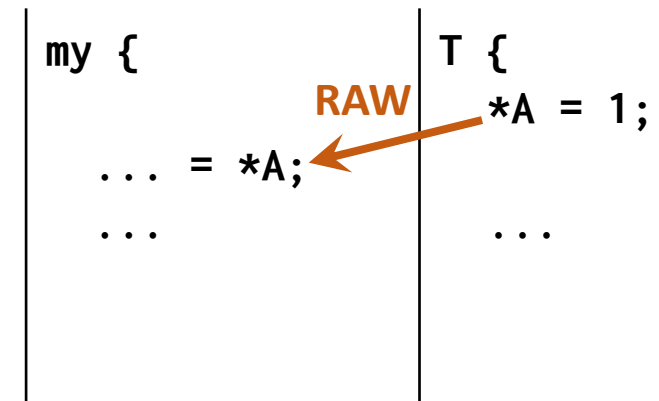
- If X is in T's Read Set: roll back either transaction
- If X is in T's Write Set: roll back either transaction

Upon a load instruction:

Add address X to Read Set (if not already there)

For each in-flight transaction T:

- If X is in T's Write Set: **roll back either transaction?**



Eager Conflict Detection – Eager VM

Upon committing transaction:

Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

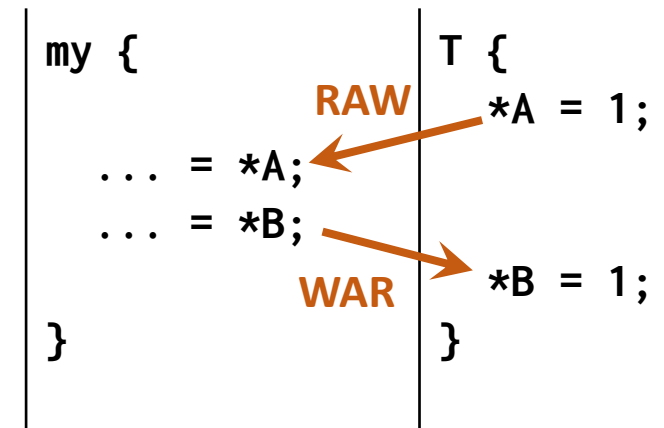
- If X is in T's Read Set: roll back either transaction
- If X is in T's Write Set: roll back either transaction

Upon a load instruction:

Add address X to Read Set (if not already there)

For each in-flight transaction T:

- If X is in T's Write Set: **roll back either transaction?**



Eager Conflict Detection – Eager VM

Upon committing transaction:

Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

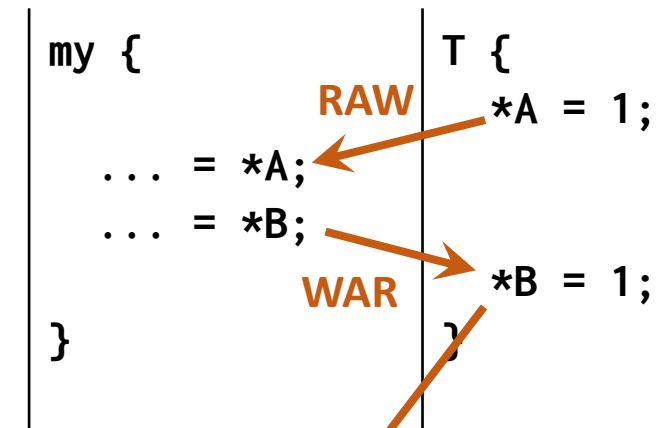
- If X is in T's Read Set: roll back either transaction
- If X is in T's Write Set: roll back either transaction

Upon a load instruction:

Add address X to Read Set (if not already there)

For each in-flight transaction T:

- If X is in T's Write Set: **roll back either transaction?**



Eager Conflict Detection – Eager VM

Upon committing transaction:

Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

- If X is in T's Read Set: roll back either transaction
- If X is in T's Write Set: roll back either transaction

Upon a load instruction:

Add address X to Read Set (if not already there)

For each in-flight transaction T:

- If X is in T's Write Set: **roll back either transaction?**

my {	T {
	*A = 1;
... = *A;	
... = *B;	
}	
	*B = 1;
	}

Eager Conflict Detection – Eager VM

Upon committing transaction:

Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

- If X is in T's Read Set: roll back either transaction
- If X is in T's Write Set: roll back either transaction

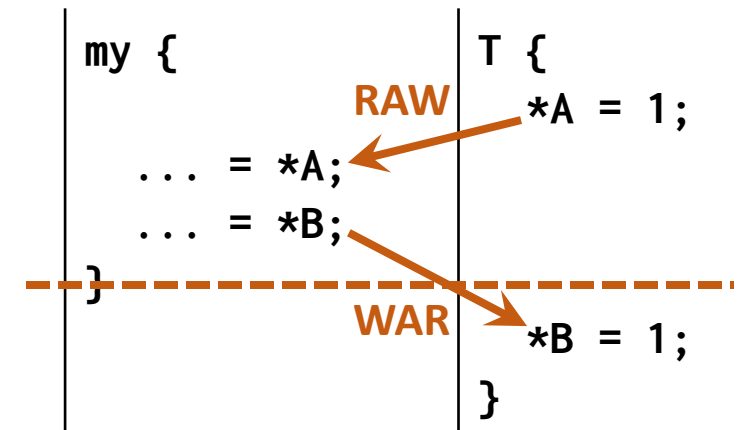
Upon a load instruction:

Add address X to Read Set (if not already there)

For each in-flight transaction T:

- If X is in T's Write Set: **roll back either transaction**

may read new A but old B,
before T can detect it



must roll back if there's a chance of
undetected interdependence



Eager Conflict Detection – Eager VM

Upon committing transaction:

Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

- If X is in T's Read Set: roll back either transaction
- If X is in T's Write Set: roll back either transaction

Upon a load instruction:

Add address X to Read Set (if not already there)

For each in-flight transaction T:

- If X is in T's Write Set: roll back either transaction

Eager Conflict Detection – Lazy VM

Upon committing transaction:

Clear Write Set and Read Set

Eager Conflict Detection – Lazy VM

Upon committing transaction:

Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

Eager Conflict Detection – Lazy VM

Upon committing transaction:

Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

➤ If X is in T's Read Set: **roll back either transaction?**

my {	T {
*A = 1;	... = *A;
*B = 1;	... = *B;
}	}

Eager Conflict Detection – Lazy VM

Upon committing transaction:

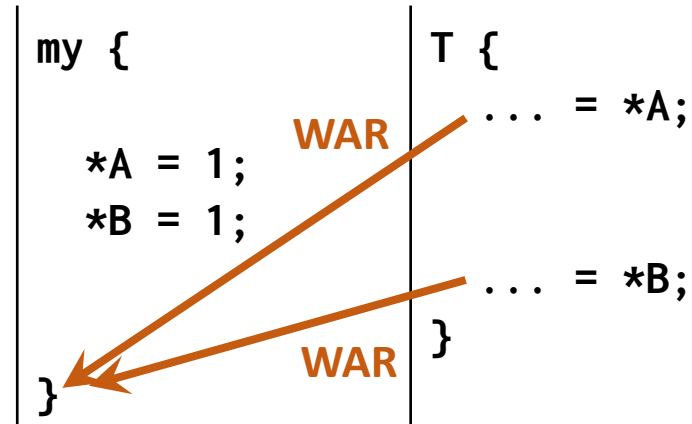
Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

➤ If X is in T's Read Set: **roll back either transaction?**



Eager Conflict Detection – Lazy VM

Upon committing transaction:

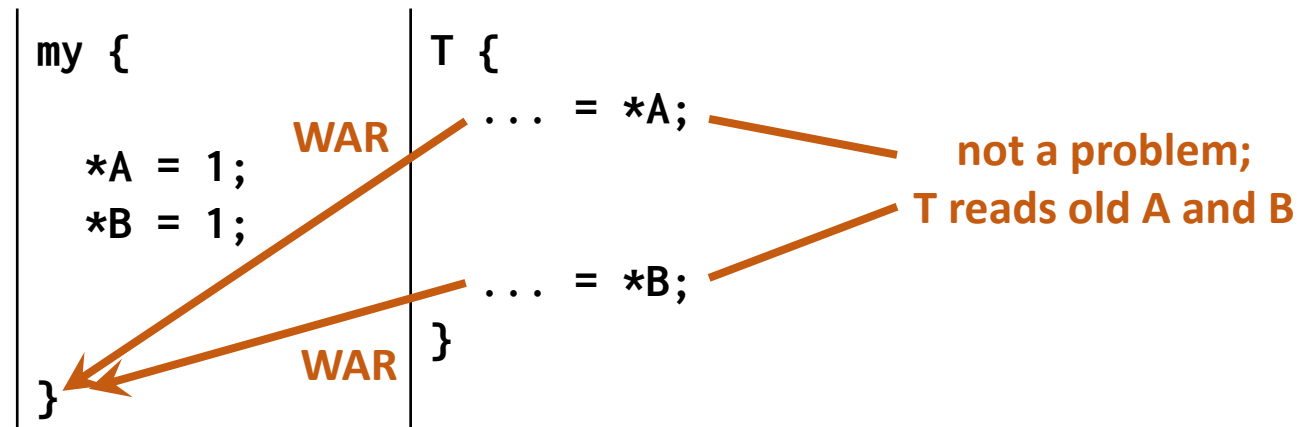
Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

➤ If X is in T's Read Set: **roll back either transaction?**



Eager Conflict Detection – Lazy VM

Upon committing transaction:

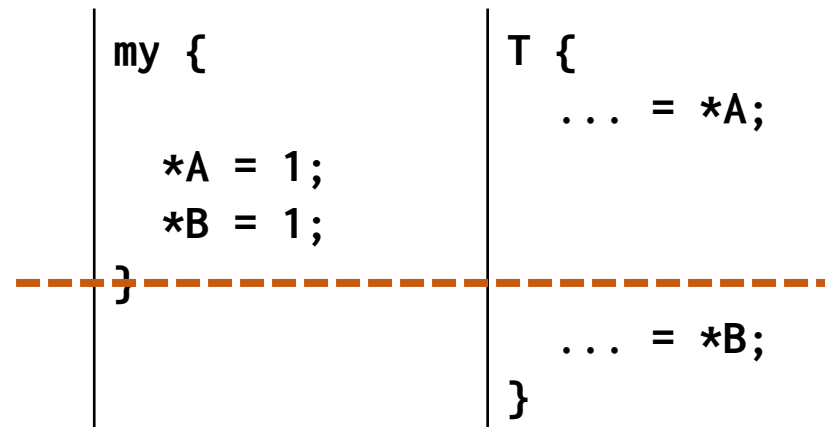
Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

➤ If X is in T's Read Set: **roll back either transaction?**



Eager Conflict Detection – Lazy VM

Upon committing transaction:

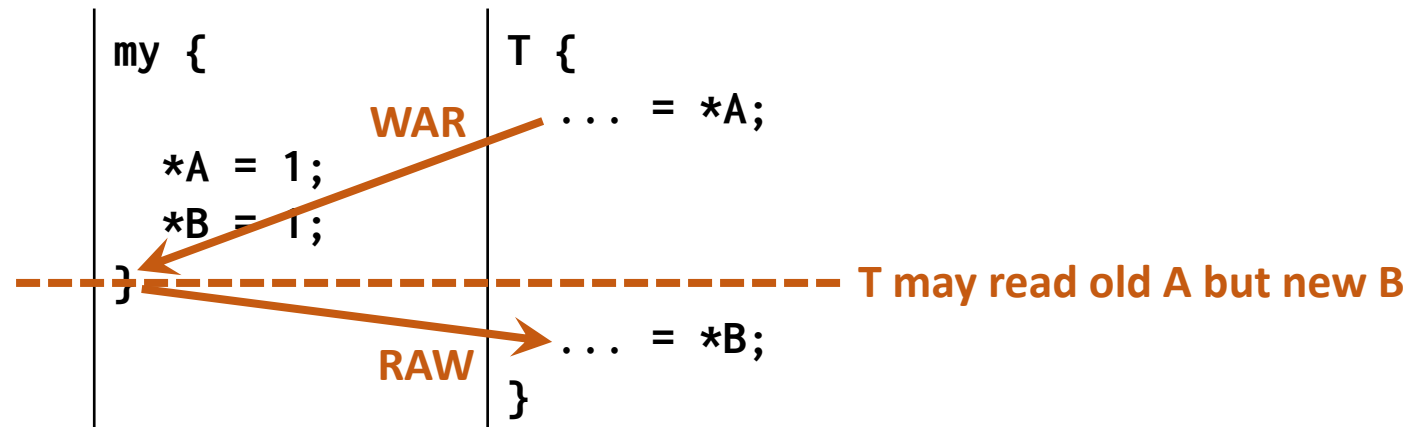
Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

➤ If X is in T's Read Set: **roll back either transaction**



Eager Conflict Detection – Lazy VM

Upon committing transaction:

Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

- If X is in T's Read Set: roll back either transaction
- If X is in T's Write Set: **roll back either transaction?**



Eager Conflict Detection – Lazy VM

Upon committing transaction:

Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

- If X is in T's Read Set: roll back either transaction
- If X is in T's Write Set: **roll back either transaction?**

```
my {  
    *A = 0;  
    ...  
}
```

```
T {  
    *A = 1;  
    ...  
}
```



Eager Conflict Detection – Lazy VM

Upon committing transaction:

Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

- If X is in T's Read Set: roll back either transaction
- If X is in T's Write Set: **roll back either transaction?**

```
my {  
    *A = 0;  
    *B = 0;  
}
```

```
T {  
    *A = 1;  
  
    *B = 1;  
}
```



Eager Conflict Detection – Lazy VM

Upon committing transaction:

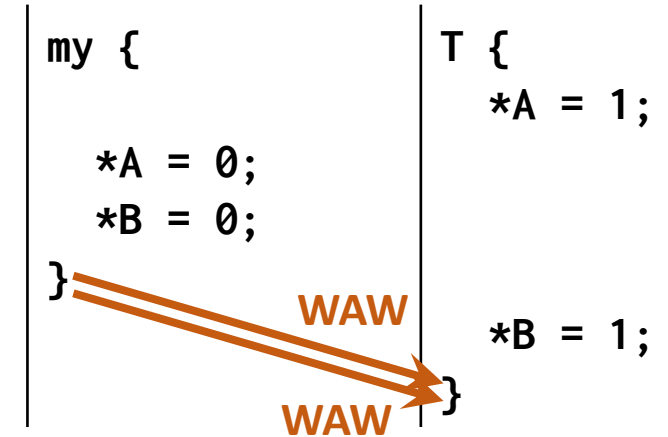
Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

- If X is in T's Read Set: roll back either transaction
- If X is in T's Write Set: **roll back either transaction?**



Eager Conflict Detection – Lazy VM

Upon committing transaction:

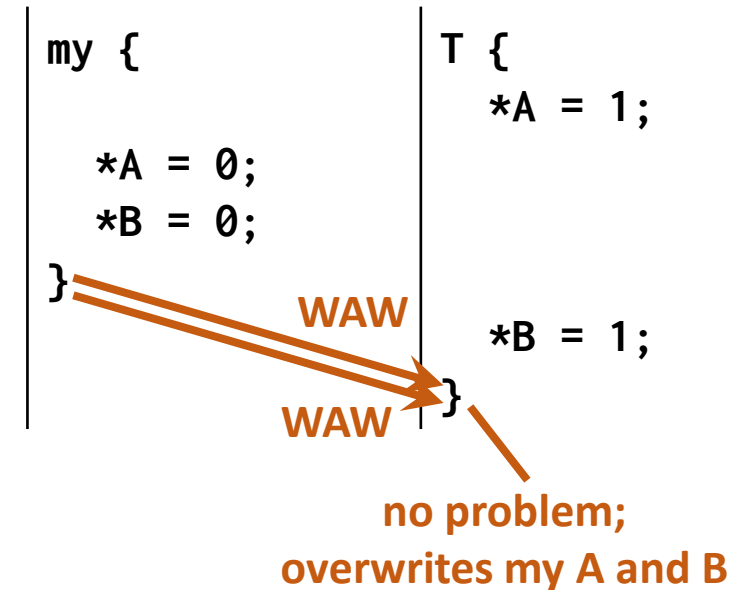
Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

- If X is in T's Read Set: roll back either transaction
- If X is in T's Write Set: **roll back either transaction?**



Eager Conflict Detection – Lazy VM

Upon committing transaction:

Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

- If X is in T's Read Set: roll back either transaction
- If X is in T's Write Set: **roll back either transaction?**

<pre>my { *A = 0; *B = 0; }</pre>	<pre>T { *A = 1; *B = 1; }</pre>
---	---



Eager Conflict Detection – Lazy VM

Upon committing transaction:

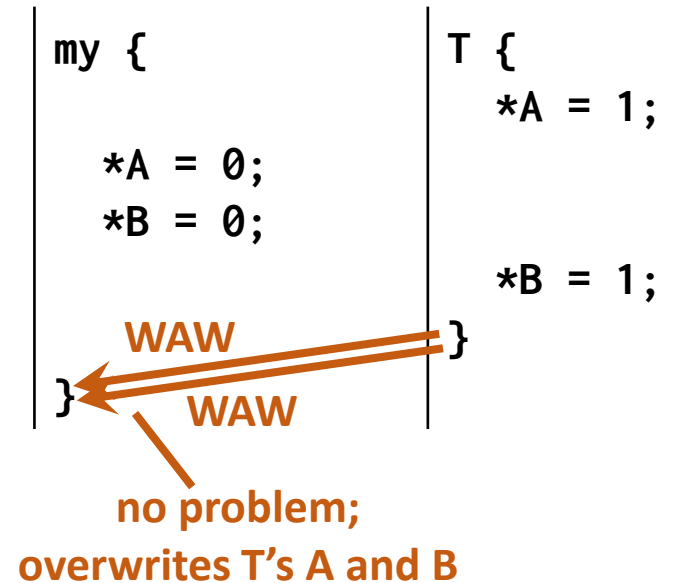
Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

- If X is in T's Read Set: roll back either transaction
- If X is in T's Write Set: **roll back either transaction?**



Eager Conflict Detection – Lazy VM

Upon committing transaction:

Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

- If X is in T's Read Set: roll back either transaction
- If X is in T's Write Set: **roll back either transaction?**

```
my {  
  
    *A = 0;  
}
```

```
T {  
    *A = 1;  
    *B = 1;  
  
    if(*A == *B)  
        ...  
}
```



Eager Conflict Detection – Lazy VM

Upon committing transaction:

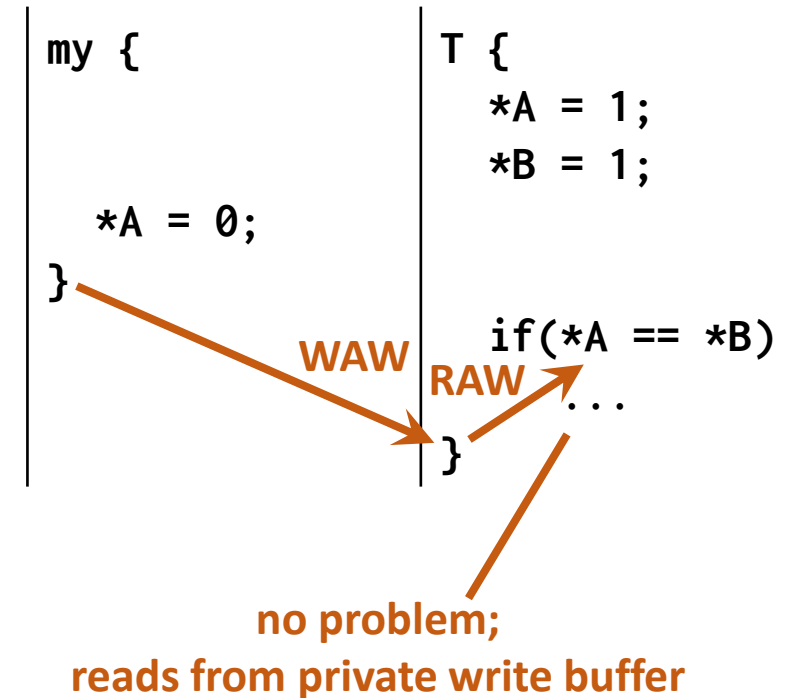
Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

- If X is in T's Read Set: roll back either transaction
- If X is in T's Write Set: **do nothing**



Eager Conflict Detection – Lazy VM

Upon committing transaction:

Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

- If X is in T's Read Set: roll back either transaction
- If X is in T's Write Set: do nothing

Upon a load instruction:

Add address X to Read Set (if not already there)

For each in-flight transaction T:

- If X is in T's Write Set: **roll back either transaction?**

Eager Conflict Detection – Lazy VM

Upon committing transaction:

Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

- If X is in T's Read Set: roll back either transaction
- If X is in T's Write Set: do nothing

Upon a load instruction:

Add address X to Read Set (if not already there)

For each in-flight transaction T:

- If X is in T's Write Set: **roll back either transaction?**

<pre>my { if(*A == *B) ... }</pre>	<pre>T { *A = 1; *B = 1; }</pre>
--	---



Eager Conflict Detection – Lazy VM

Upon committing transaction:

Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

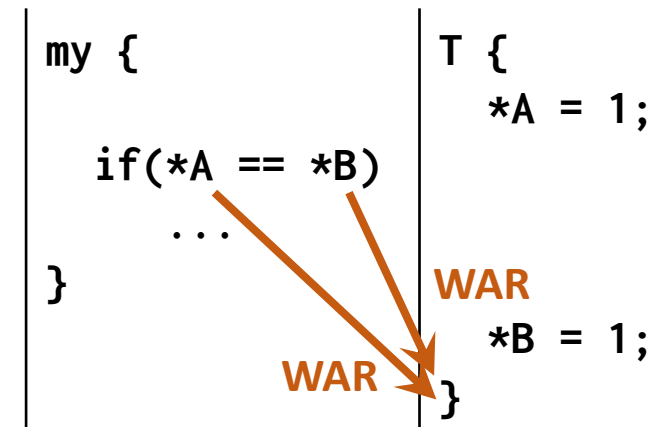
- If X is in T's Read Set: roll back either transaction
- If X is in T's Write Set: do nothing

Upon a load instruction:

Add address X to Read Set (if not already there)

For each in-flight transaction T:

- If X is in T's Write Set: **roll back either transaction?**



Eager Conflict Detection – Lazy VM

Upon committing transaction:

Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

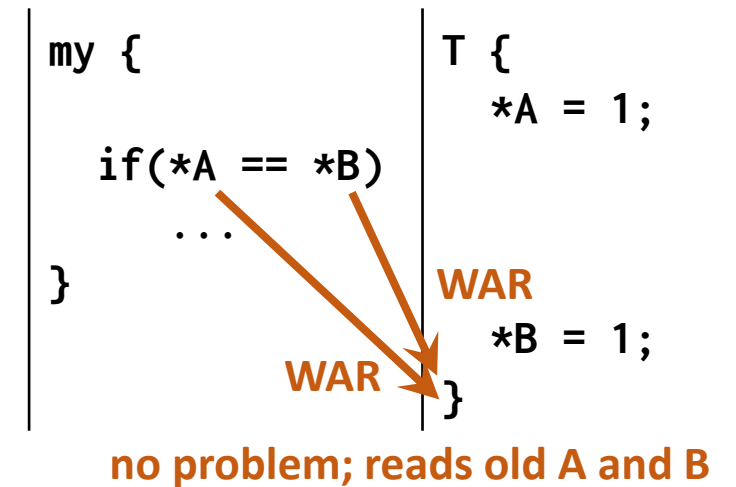
- If X is in T's Read Set: roll back either transaction
- If X is in T's Write Set: do nothing

Upon a load instruction:

Add address X to Read Set (if not already there)

For each in-flight transaction T:

- If X is in T's Write Set: **do nothing?**



Eager Conflict Detection – Lazy VM

Upon committing transaction:

Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

- If X is in T's Read Set: roll back either transaction
- If X is in T's Write Set: do nothing

Upon a load instruction:

Add address X to Read Set (if not already there)

For each in-flight transaction T:

- If X is in T's Write Set: **do nothing?**

<pre>my { if(*A... ...== *B) ... }</pre>	<pre>T { *A = 1; *B = 1; }</pre>
--	--

Eager Conflict Detection – Lazy VM

Upon committing transaction:

Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

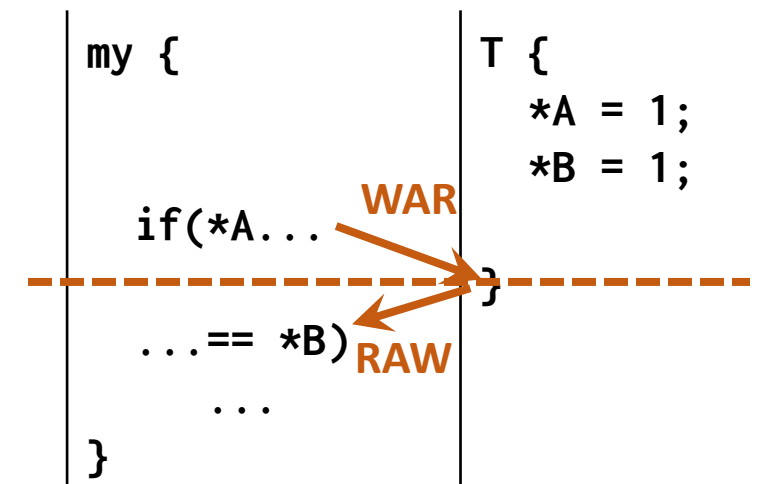
- If X is in T's Read Set: roll back either transaction
- If X is in T's Write Set: do nothing

Upon a load instruction:

Add address X to Read Set (if not already there)

For each in-flight transaction T:

- If X is in T's Write Set: **do nothing?**



may read old A but new B

Eager Conflict Detection – Lazy VM

Upon committing transaction:

Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

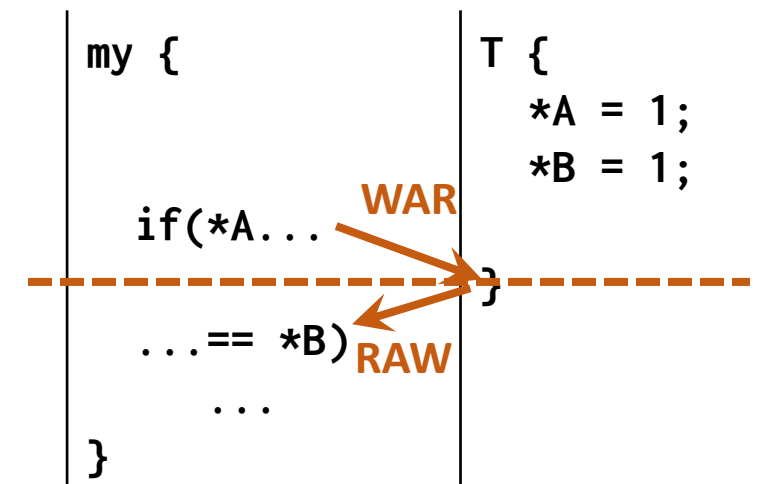
- If X is in T's Read Set: roll back either transaction
- If X is in T's Write Set: do nothing

Upon a load instruction:

Add address X to Read Set (if not already there)

For each in-flight transaction T:

- If X is in T's Write Set: **roll back either transaction**



may read old A but new B

Eager Conflict Detection – Lazy VM

Upon committing transaction:

Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

- If X is in T's Read Set: roll back either transaction
- If X is in T's Write Set: do nothing

Upon a load instruction:

Add address X to Read Set (if not already there)

For each in-flight transaction T:

- If X is in T's Write Set: roll back either transaction

Lazy Conflict Detection – Lazy VM

Upon a store instruction:

Add address X to Write Set (if not already there)

Upon a load instruction:

Add address X to Read Set (if not already there)

Lazy Conflict Detection – Lazy VM

Upon a store instruction:

Add address X to Write Set (if not already there)

Upon a load instruction:

Add address X to Read Set (if not already there)

Upon committing transaction:

For each address X in Write Set and each in-flight transaction T:

➤ If X is in T's Read Set: **roll back either transaction?**

Lazy Conflict Detection – Lazy VM

Upon a store instruction:

Add address X to Write Set (if not already there)

Upon a load instruction:

Add address X to Read Set (if not already there)

my {	T {
*A = 1;	... = *A;
*B = 1;	...
}	

Upon committing transaction:

For each address X in Write Set and each in-flight transaction T:

➤ If X is in T's Read Set: **roll back either transaction?**

Lazy Conflict Detection – Lazy VM

Upon a store instruction:

Add address X to Write Set (if not already there)

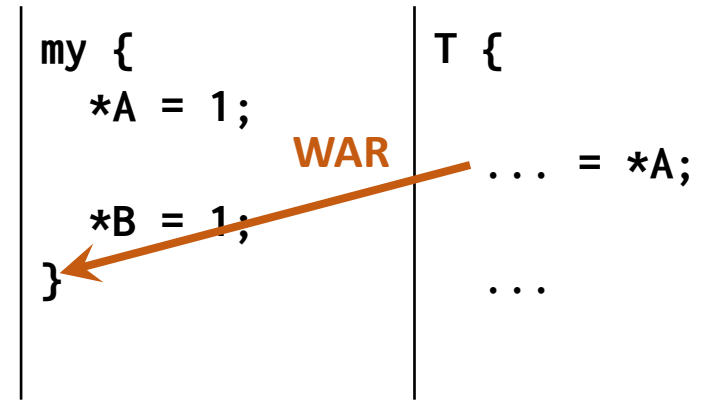
Upon a load instruction:

Add address X to Read Set (if not already there)

Upon committing transaction:

For each address X in Write Set and each in-flight transaction T:

➤ If X is in T's Read Set: **roll back either transaction?**



Lazy Conflict Detection – Lazy VM

Upon a store instruction:

Add address X to Write Set (if not already there)

Upon a load instruction:

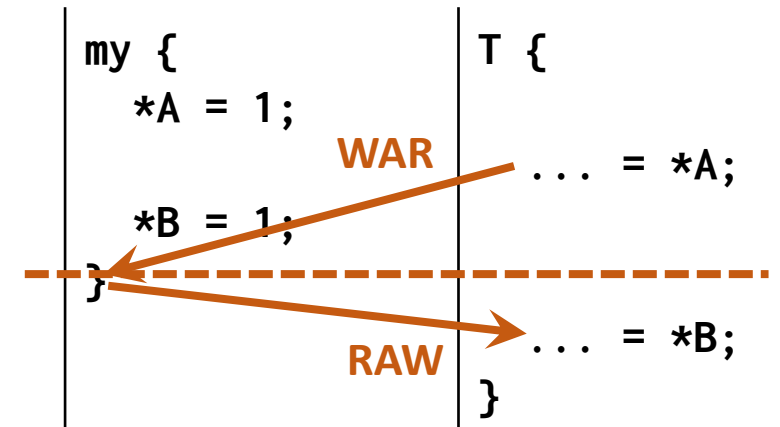
Add address X to Read Set (if not already there)

Upon committing transaction:

For each address X in Write Set and each in-flight transaction T:

➤ If X is in T's Read Set: **roll back either transaction**

better to roll back T's
transaction, since mine is closer
to finish line



T may read old A but new B

Lazy Conflict Detection – Lazy VM

Upon a store instruction:

Add address X to Write Set (if not already there)

Upon a load instruction:

Add address X to Read Set (if not already there)

Upon committing transaction:

For each address X in Write Set and each in-flight transaction T:

- If X is in T's Read Set: roll back either transaction
- If X is in T's Write Set: **roll back either transaction?**

Lazy Conflict Detection – Lazy VM

Upon a store instruction:

Add address X to Write Set (if not already there)

Upon a load instruction:

Add address X to Read Set (if not already there)

Upon committing transaction:

For each address X in Write Set and each in-flight transaction T:

- If X is in T's Read Set: roll back either transaction
- If X is in T's Write Set: **roll back either transaction?**

```
my {  
  
    *A = 0;  
}
```

```
T {  
    *A = 1;  
    *B = 1;  
  
    if(*A == *B)  
        ...  
}
```



Lazy Conflict Detection – Lazy VM

Upon a store instruction:

Add address X to Write Set (if not already there)

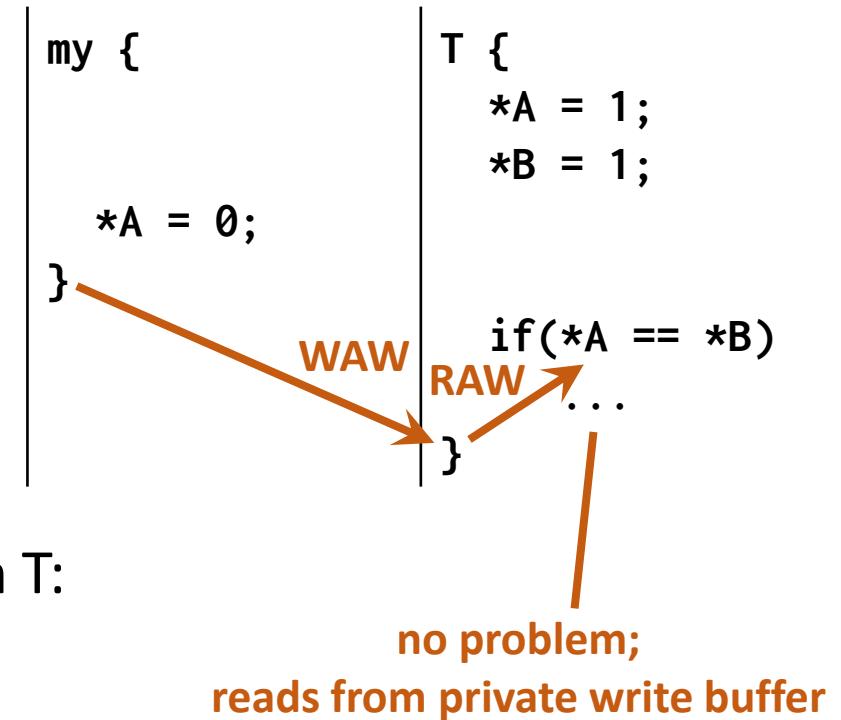
Upon a load instruction:

Add address X to Read Set (if not already there)

Upon committing transaction:

For each address X in Write Set and each in-flight transaction T:

- If X is in T's Read Set: roll back either transaction
- If X is in T's Write Set: **roll back either transaction?**



Lazy Conflict Detection – Lazy VM

Upon a store instruction:

Add address X to Write Set (if not already there)

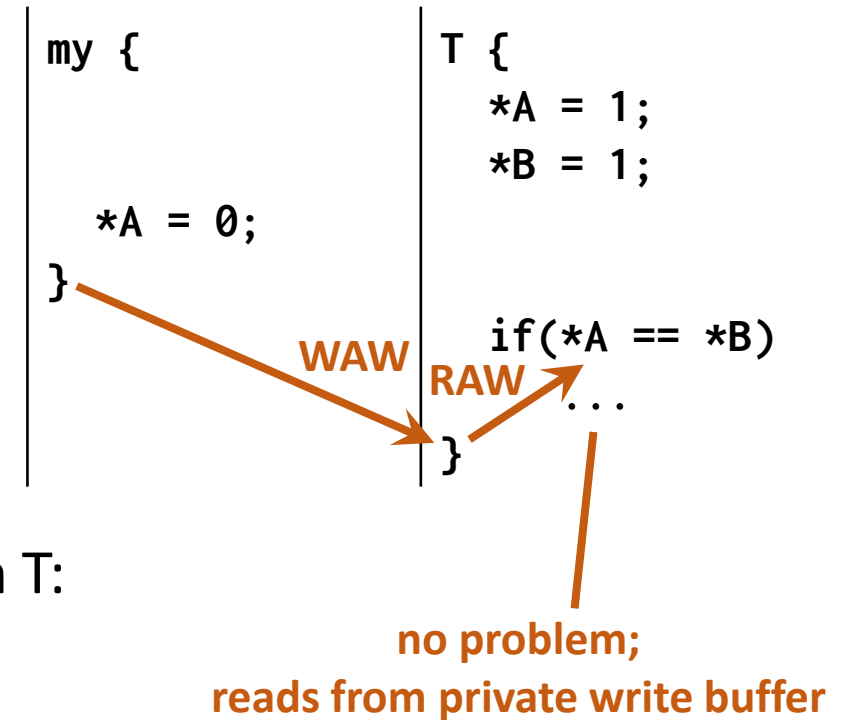
Upon a load instruction:

Add address X to Read Set (if not already there)

Upon committing transaction:

For each address X in Write Set and each in-flight transaction T:

- If X is in T's Read Set: roll back either transaction
- If X is in T's Write Set: **do nothing**



Lazy Conflict Detection – Lazy VM

Upon a store instruction:

Add address X to Write Set (if not already there)

Upon a load instruction:

Add address X to Read Set (if not already there)

Upon committing transaction:

For each address X in Write Set and each in-flight transaction T:

- If X is in T's Read Set: roll back either transaction
- If X is in T's Write Set: do nothing

For each address X in Read Set and each in-flight transaction T:

- If X is in T's Write Set: **roll back either transaction?**



Lazy Conflict Detection – Lazy VM

Upon a store instruction:

Add address X to Write Set (if not already there)

Upon a load instruction:

Add address X to Read Set (if not already there)

<pre>my { ... = *A; ... = *B; }</pre>	<pre>T { *A = 1; ...</pre>
--	---

Upon committing transaction:

For each address X in Write Set and each in-flight transaction T:

- If X is in T's Read Set: roll back either transaction
- If X is in T's Write Set: do nothing

For each address X in Read Set and each in-flight transaction T:

- If X is in T's Write Set: **roll back either transaction?**

Lazy Conflict Detection – Lazy VM

Upon a store instruction:

Add address X to Write Set (if not already there)

Upon a load instruction:

Add address X to Read Set (if not already there)

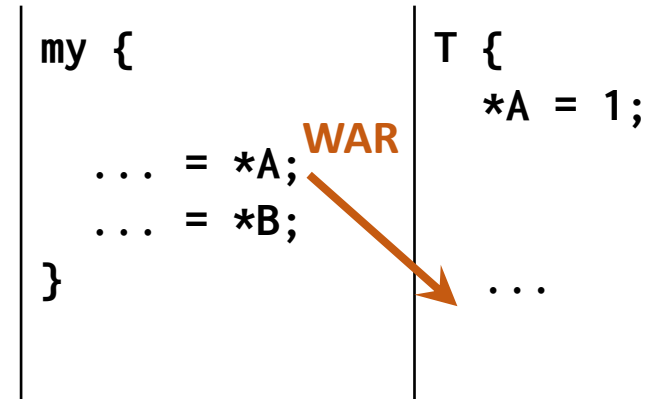
Upon committing transaction:

For each address X in Write Set and each in-flight transaction T:

- If X is in T's Read Set: roll back either transaction
- If X is in T's Write Set: do nothing

For each address X in Read Set and each in-flight transaction T:

- If X is in T's Write Set: **roll back either transaction?**



Lazy Conflict Detection – Lazy VM

Upon a store instruction:

Add address X to Write Set (if not already there)

Upon a load instruction:

Add address X to Read Set (if not already there)

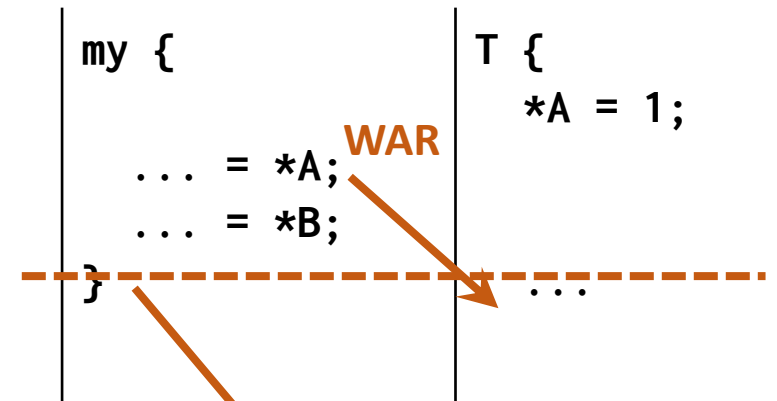
Upon committing transaction:

For each address X in Write Set and each in-flight transaction T:

- If X is in T's Read Set: roll back either transaction
- If X is in T's Write Set: do nothing

For each address X in Read Set and each in-flight transaction T:

- If X is in T's Write Set: **do nothing**



at this point, it's impossible for my transaction to be dependent on T



Lazy Conflict Detection – Lazy VM

Upon a store instruction:

Add address X to Write Set (if not already there)

Upon a load instruction:

Add address X to Read Set (if not already there)

Upon committing transaction:

For each address X in Write Set and each in-flight transaction T:

- If X is in T's Read Set: roll back either transaction
- If X is in T's Write Set: do nothing

For each address X in Read Set and each in-flight transaction T:

- If X is in T's Write Set: do nothing

Clear Write Set and Read Set

Lazy Conflict Detection – Lazy VM

Upon a store instruction:

Add address X to Write Set (if not already there)

Upon a load instruction:

Add address X to Read Set (if not already there)

Upon committing transaction:

For each address X in Write Set and each in-flight transaction T:

- If X is in T's Read Set: roll back either transaction
- If X is in T's Write Set: do nothing

writer's responsibility to roll back readers when committing

For each address X in Read Set and each in-flight transaction T:

- If X is in T's Write Set: do nothing

Clear Write Set and Read Set



Lazy Conflict Detection – Eager VM

Upon a store instruction:

Add address X to Write Set (if not already there)

Upon a load instruction:

Add address X to Read Set (if not already there)

Lazy Conflict Detection – Eager VM

Upon a store instruction:

Add address X to Write Set (if not already there)

Upon a load instruction:

Add address X to Read Set (if not already there)

Upon committing transaction:

For each address X in Write Set and each in-flight transaction T:

➤ If X is in T's Read Set: **roll back either transaction?**



Lazy Conflict Detection – Eager VM

Upon a store instruction:

Add address X to Write Set (if not already there)

Upon a load instruction:

Add address X to Read Set (if not already there)

Upon committing transaction:

For each address X in Write Set and each in-flight transaction T:

➤ If X is in T's Read Set: **roll back either transaction?**

my {	T {
	... = *A;
*A = 1;	
*B = 1;	... = *B;
}	...



Lazy Conflict Detection – Eager VM

Upon a store instruction:

Add address X to Write Set (if not already there)

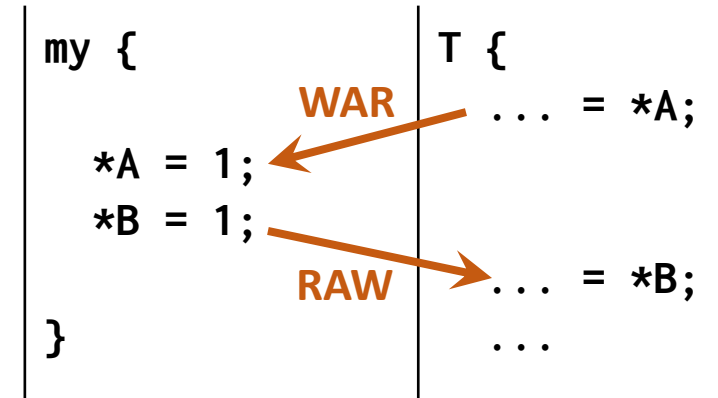
Upon a load instruction:

Add address X to Read Set (if not already there)

Upon committing transaction:

For each address X in Write Set and each in-flight transaction T:

➤ If X is in T's Read Set: **roll back either transaction?**



Lazy Conflict Detection – Eager VM

Upon a store instruction:

Add address X to Write Set (if not already there)

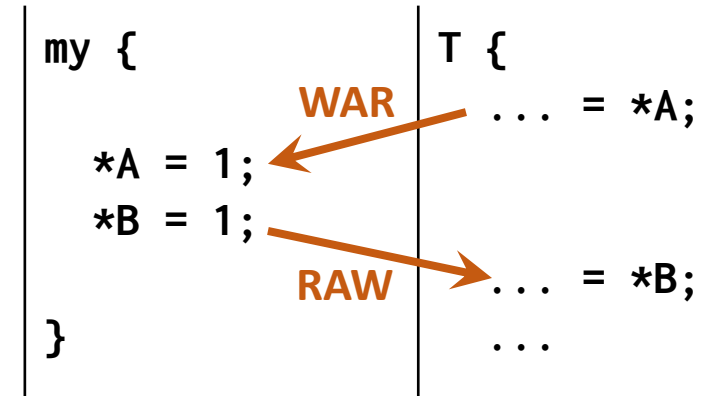
Upon a load instruction:

Add address X to Read Set (if not already there)

Upon committing transaction:

For each address X in Write Set and each in-flight transaction T:

➤ If X is in T's Read Set: **roll back either transaction?**



would rolling back T's transaction resolve the conflict?

Lazy Conflict Detection – Eager VM

Upon a store instruction:

Add address X to Write Set (if not already there)

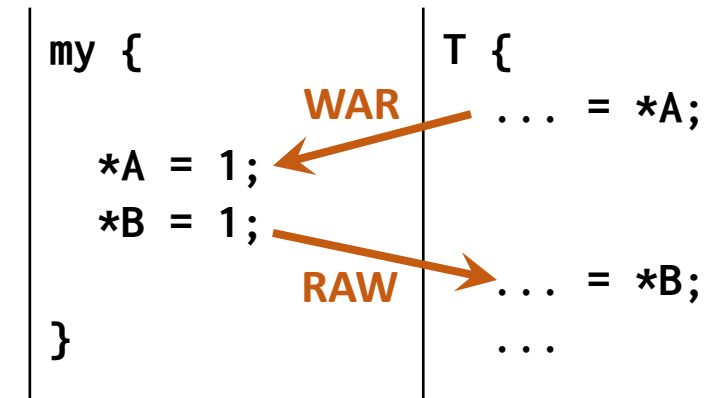
Upon a load instruction:

Add address X to Read Set (if not already there)

Upon committing transaction:

For each address X in Write Set and each in-flight transaction T:

➤ If X is in T's Read Set: **roll back either transaction?**



would rolling back T's transaction resolve the conflict? yes

Lazy Conflict Detection – Eager VM

Upon a store instruction:

Add address X to Write Set (if not already there)

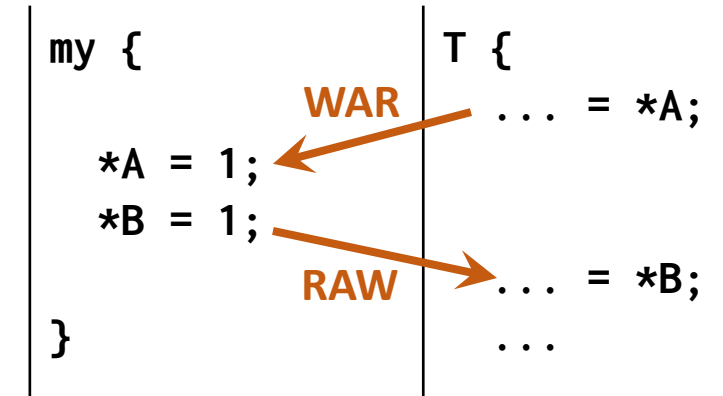
Upon a load instruction:

Add address X to Read Set (if not already there)

Upon committing transaction:

For each address X in Write Set and each in-flight transaction T:

➤ If X is in T's Read Set: **roll back either transaction?**



would rolling back T's transaction resolve the conflict? yes
would rolling back my transaction resolve the conflict?

Lazy Conflict Detection – Eager VM

Upon a store instruction:

Add address X to Write Set (if not already there)

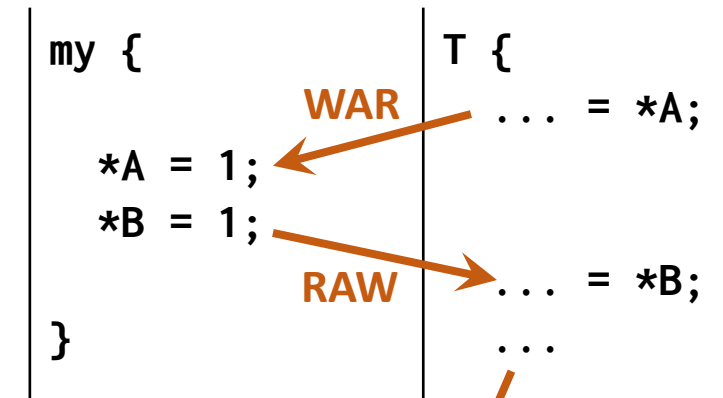
Upon a load instruction:

Add address X to Read Set (if not already there)

Upon committing transaction:

For each address X in Write Set and each in-flight transaction T:

➤ If X is in T's Read Set: **roll back either transaction?**



T already read old A and new B

would rolling back T's transaction resolve the conflict? yes
would rolling back my transaction resolve the conflict? no

Lazy Conflict Detection – Eager VM

Upon a store instruction:

Add address X to Write Set (if not already there)

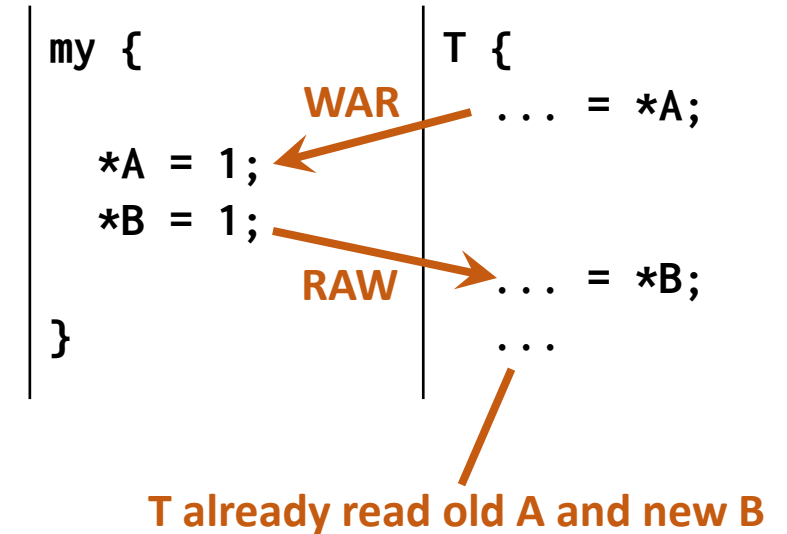
Upon a load instruction:

Add address X to Read Set (if not already there)

Upon committing transaction:

For each address X in Write Set and each in-flight transaction T:

➤ If X is in T's Read Set: **roll back T's transaction**



Lazy Conflict Detection – Eager VM

Upon a store instruction:

Add address X to Write Set (if not already there)

Upon a load instruction:

Add address X to Read Set (if not already there)

Upon committing transaction:

For each address X in Write Set and each in-flight transaction T:

- If X is in T's Read Set: roll back T's transaction
- If X is in T's Write Set: **roll back either transaction?**

Lazy Conflict Detection – Eager VM

Upon a store instruction:

Add address X to Write Set (if not already there)

Upon a load instruction:

Add address X to Read Set (if not already there)

Upon committing transaction:

For each address X in Write Set and each in-flight transaction T:

- If X is in T's Read Set: roll back T's transaction
- If X is in T's Write Set: **roll back either transaction?**

my {	T {
*A = 1;	*B = 2;
*B = 1;	*A = 2;
}	...

Lazy Conflict Detection – Eager VM

Upon a store instruction:

Add address X to Write Set (if not already there)

Upon a load instruction:

Add address X to Read Set (if not already there)

Upon committing transaction:

For each address X in Write Set and each in-flight transaction T:

- If X is in T's Read Set: roll back T's transaction
- If X is in T's Write Set: **roll back either transaction**

<pre>my { *A = 1; *B = 1; }</pre>	<pre>T { *B = 2; *A = 2; ...</pre>
--	--

**need to remember the ordering
of writes (e.g., timestamps,
version numbers);
don't undo stale writes**

Lazy Conflict Detection – Eager VM

Upon a store instruction:

Add address X to Write Set (if not already there)

Upon a load instruction:

Add address X to Read Set (if not already there)

Upon committing transaction:

For each address X in Write Set and each in-flight transaction T:

- If X is in T's Read Set: roll back T's transaction
- If X is in T's Write Set: roll back either transaction

For each address X in Read Set and each in-flight transaction T:

- If X is in T's Write Set: **roll back either transaction?**

Lazy Conflict Detection – Eager VM

Upon a store instruction:

Add address X to Write Set (if not already there)

Upon a load instruction:

Add address X to Read Set (if not already there)

<pre>my { if(*A == *B) ... }</pre>	<pre>T { *A = 1; ... }</pre>
--	--

Upon committing transaction:

For each address X in Write Set and each in-flight transaction T:

- If X is in T's Read Set: roll back T's transaction
- If X is in T's Write Set: roll back either transaction

For each address X in Read Set and each in-flight transaction T:

- If X is in T's Write Set: **roll back either transaction?**

Lazy Conflict Detection – Eager VM

Upon a store instruction:

Add address X to Write Set (if not already there)

Upon a load instruction:

Add address X to Read Set (if not already there)

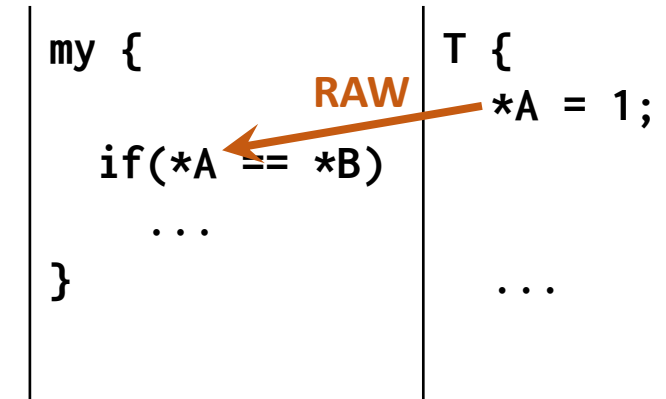
Upon committing transaction:

For each address X in Write Set and each in-flight transaction T:

- If X is in T's Read Set: roll back T's transaction
- If X is in T's Write Set: roll back either transaction

For each address X in Read Set and each in-flight transaction T:

- If X is in T's Write Set: **roll back either transaction?**



Lazy Conflict Detection – Eager VM

Upon a store instruction:

Add address X to Write Set (if not already there)

Upon a load instruction:

Add address X to Read Set (if not already there)

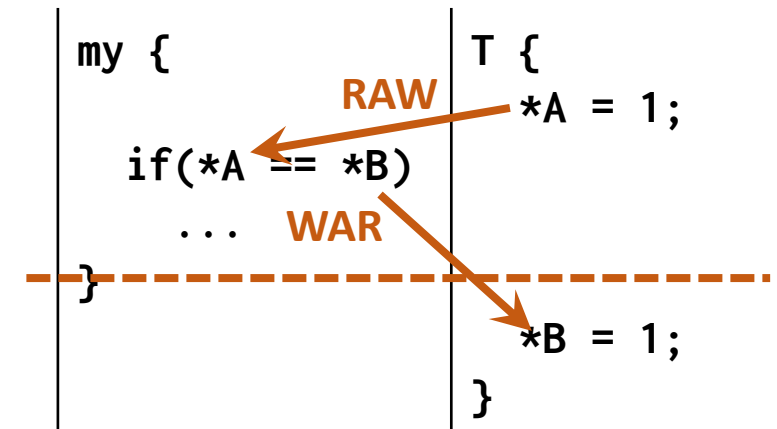
Upon committing transaction:

For each address X in Write Set and each in-flight transaction T:

- If X is in T's Read Set: roll back T's transaction
- If X is in T's Write Set: roll back either transaction

For each address X in Read Set and each in-flight transaction T:

- If X is in T's Write Set: **roll back either transaction?**



Lazy Conflict Detection – Eager VM

Upon a store instruction:

Add address X to Write Set (if not already there)

Upon a load instruction:

Add address X to Read Set (if not already there)

Upon committing transaction:

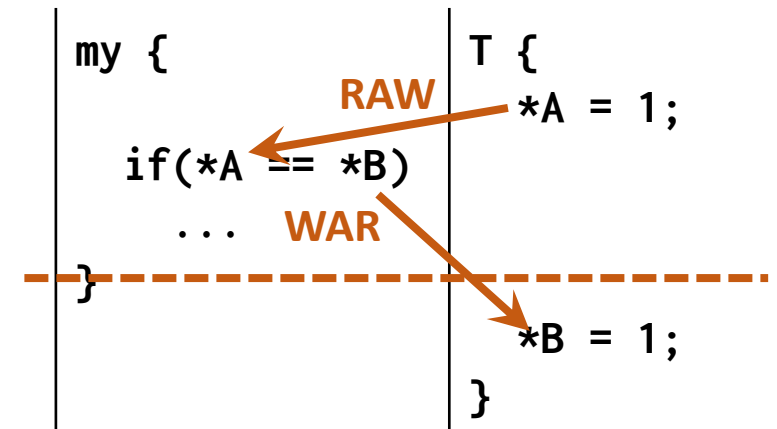
For each address X in Write Set and each in-flight transaction T:

- If X is in T's Read Set: roll back T's transaction
- If X is in T's Write Set: roll back either transaction

For each address X in Read Set and each in-flight transaction T:

- If X is in T's Write Set: **roll back either transaction?**

would rolling back my transaction resolve the conflict?



Lazy Conflict Detection – Eager VM

Upon a store instruction:

Add address X to Write Set (if not already there)

Upon a load instruction:

Add address X to Read Set (if not already there)

Upon committing transaction:

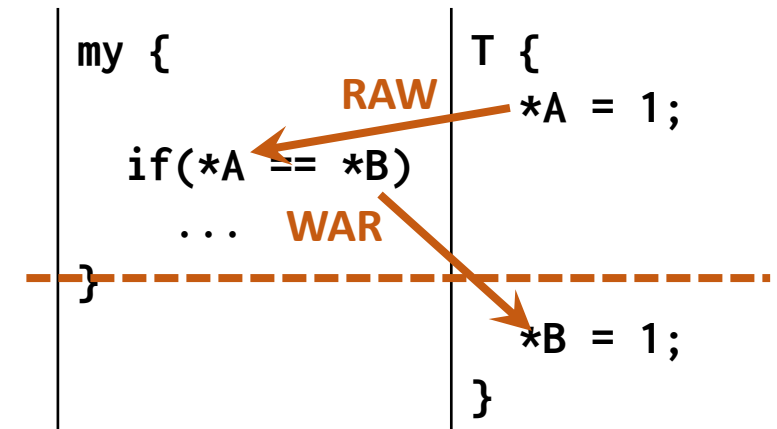
For each address X in Write Set and each in-flight transaction T:

- If X is in T's Read Set: roll back T's transaction
- If X is in T's Write Set: roll back either transaction

For each address X in Read Set and each in-flight transaction T:

- If X is in T's Write Set: **roll back either transaction?**

would rolling back my transaction resolve the conflict? yes



Lazy Conflict Detection – Eager VM

Upon a store instruction:

Add address X to Write Set (if not already there)

Upon a load instruction:

Add address X to Read Set (if not already there)

Upon committing transaction:

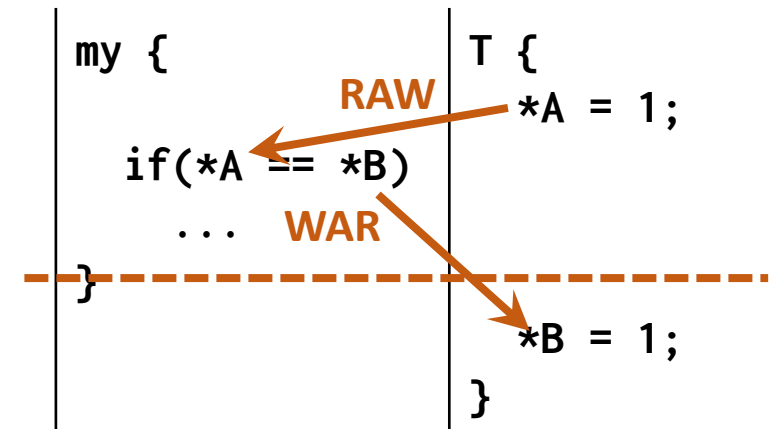
For each address X in Write Set and each in-flight transaction T:

- If X is in T's Read Set: roll back T's transaction
- If X is in T's Write Set: roll back either transaction

For each address X in Read Set and each in-flight transaction T:

- If X is in T's Write Set: **roll back either transaction?**

would rolling back my transaction resolve the conflict? yes
would rolling back T's transaction resolve the conflict?



Lazy Conflict Detection – Eager VM

Upon a store instruction:

Add address X to Write Set (if not already there)

Upon a load instruction:

Add address X to Read Set (if not already there)

Upon committing transaction:

For each address X in Write Set and each in-flight transaction T:

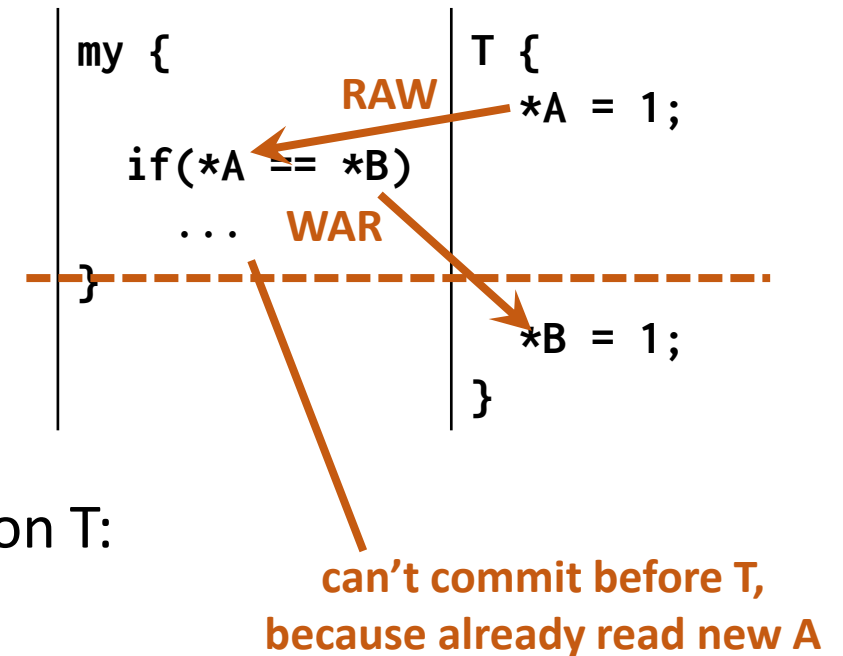
- If X is in T's Read Set: roll back T's transaction
- If X is in T's Write Set: roll back either transaction

For each address X in Read Set and each in-flight transaction T:

- If X is in T's Write Set: **roll back either transaction?**

would rolling back my transaction resolve the conflict? yes

would rolling back T's transaction resolve the conflict? no



Lazy Conflict Detection – Eager VM

Upon a store instruction:

Add address X to Write Set (if not already there)

Upon a load instruction:

Add address X to Read Set (if not already there)

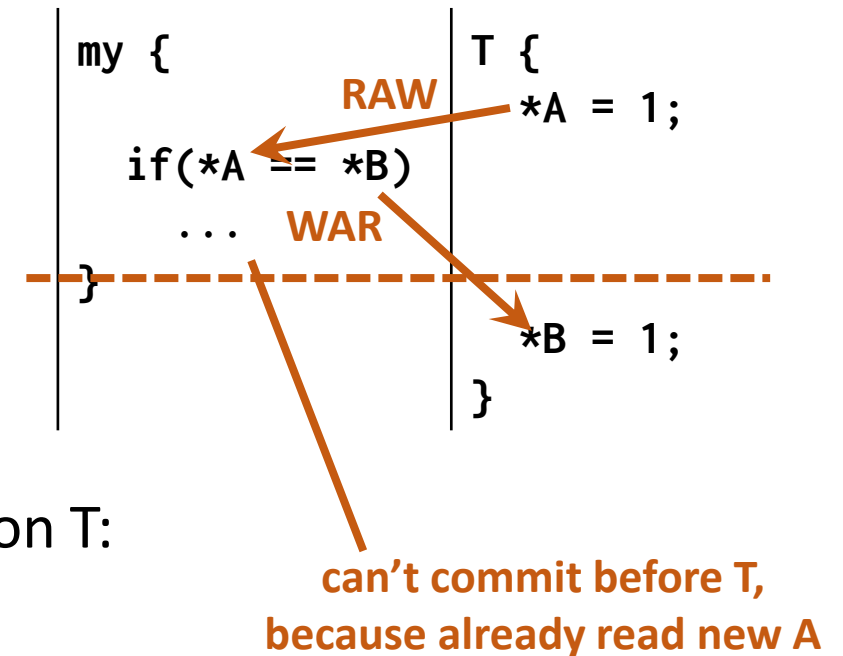
Upon committing transaction:

For each address X in Write Set and each in-flight transaction T:

- If X is in T's Read Set: roll back T's transaction
- If X is in T's Write Set: roll back either transaction

For each address X in Read Set and each in-flight transaction T:

- If X is in T's Write Set: **roll back my transaction**



Lazy Conflict Detection – Eager VM

Upon a store instruction:

Add address X to Write Set (if not already there)

Upon a load instruction:

Add address X to Read Set (if not already there)

Upon committing transaction:

For each address X in Write Set and each in-flight transaction T:

- If X is in T's Read Set: roll back T's transaction
- If X is in T's Write Set: roll back either transaction

For each address X in Read Set and each in-flight transaction T:

- If X is in T's Write Set: roll back my transaction

Clear Write Set and Read Set

Lazy Conflict Detection – Eager VM

Upon a store instruction:

Add address X to Write Set (if not already there)

Upon a load instruction:

Add address X to Read Set (if not already there)

Upon committing transaction:

For each address X in Write Set and each in-flight transaction T:

- If X is in T's Read Set: roll back T's transaction
- If X is in T's Write Set: roll back either transaction

For each address X in Read Set and each in-flight transaction T:

- If X is in T's Write Set: roll back my transaction

Clear Write Set and Read Set

always roll back the reader



Lazy Conflict Detection – Eager VM

Upon a store instruction:

Add address X to Write Set (if not already there)

Upon a load instruction:

Add address X to Read Set (if not already there)

Upon committing transaction:

For each address X in Write Set and each in-flight transaction T:

- If X is in T's Read Set: roll back T's transaction
- If X is in T's Write Set: roll back either transaction

For each address X in Read Set and each in-flight transaction T:

- If X is in T's Write Set: roll back my transaction

Clear Write Set and Read Set

<pre>my { *A = 1; if(*C == 1) ... *B = 1; }</pre>	<pre>T { *C = 1; if(*A == *B) </pre>
---	---



Lazy Conflict Detection – Eager VM

Upon a store instruction:

Add address X to Write Set (if not already there)

Upon a load instruction:

Add address X to Read Set (if not already there)

Upon committing transaction:

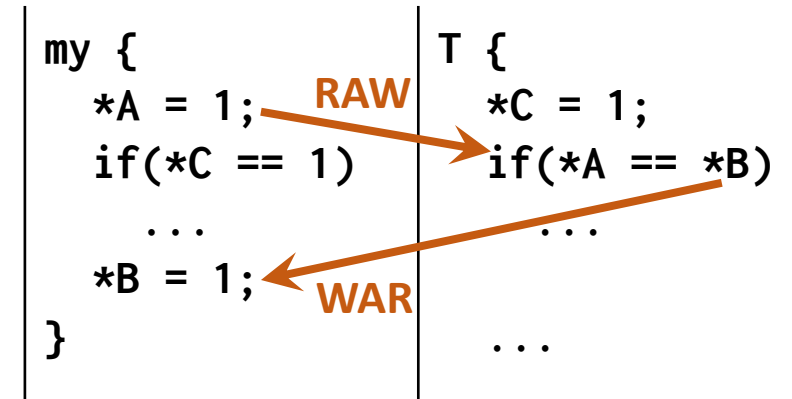
For each address X in Write Set and each in-flight transaction T:

- If X is in T's Read Set: roll back T's transaction
- If X is in T's Write Set: roll back either transaction

For each address X in Read Set and each in-flight transaction T:

- If X is in T's Write Set: roll back my transaction

Clear Write Set and Read Set



roll back T's transaction since
T read new A and old B



Lazy Conflict Detection – Eager VM

Upon a store instruction:

Add address X to Write Set (if not already there)

Upon a load instruction:

Add address X to Read Set (if not already there)

Upon committing transaction:

For each address X in Write Set and each in-flight transaction T:

- If X is in T's Read Set: roll back T's transaction
- If X is in T's Write Set: roll back either transaction

roll back T's transaction since
T read new A and old B

For each address X in Read Set and each in-flight transaction T:

- If X is in T's Write Set: roll back my transaction

but also roll back my transaction
since already read new C

Clear Write Set and Read Set

<pre>my { *A = 1; RAW if(*C == 1) ... *B = 1; }</pre>	<pre>T { *C = 1; if(*A == *B) </pre>
--	---



Lazy Conflict Detection – Eager VM

Upon a store instruction:

Add address X to Write Set (if not already there)

Upon a load instruction:

Add address X to Read Set (if not already there)

<pre>my { *A = 1; if(*C == 1) ... *B = 1; }</pre>	<pre>T { *C = 1; if(*A == *B) </pre>
---	---

Upon committing transaction:

For each address X in Write Set and each in-flight transaction T:

- If X is in T's Read Set: **flag** T's transaction
- If X is in T's Write Set: **flag** either transaction

For each address X in Read Set and each in-flight transaction T:

- If X is in T's Write Set: **flag** my transaction

Roll back flagged transactions

Clear Write Set and Read Set

Lazy Conflict Detection – Eager VM

Upon a store instruction:

Add address X to Write Set (if not already there)

Upon a load instruction:

Add address X to Read Set (if not already there)

Upon committing transaction:

For each address X in Write Set and each in-flight transaction T:

- If X is in T's Read Set: flag T's transaction
- If X is in T's Write Set: flag either transaction

For each address X in Read Set and each in-flight transaction T:

- If X is in T's Write Set: flag my transaction

Roll back flagged transactions

Clear Write Set and Read Set

Eager vs. Lazy Conflict Detection

When to use Eager

- Few load and store instructions (avoid overhead of frequent eager checks)
- High conflict rate expected (catch conflicts as early as possible)

VS

When to use Lazy

- Small memory footprint within transactions (avoid complexity of read/write set matching at commit)
- Low conflict rate expected (progress faster since conflicts are unlikely)

Eager vs. Lazy Conflict Detection

When to use Eager

- Few load and store instructions (avoid overhead of frequent eager checks)
- High conflict rate expected (catch conflicts as early as possible)

VS

When to use Lazy

- Small memory footprint within transactions (avoid complexity of read/write set matching at commit)
- **Low conflict rate expected (progress faster since conflicts are unlikely)**

key motivation for optimistic concurrency
in the first place

Will revisit later...

